

Hyperbridge MMR Audit

Hacking assessment report

V1.2, May 12th, 2026

Cayo Fletcher-Smith	cayo@srlabs.de
Jonas Panizza	jonas@srlabs.de
Rachna Shriwas	rachna@srlabs.de
Regina Biró	regina@srlabs.de

Abstract.

SRLabs audited Hyperbridge's Solidity verification and settlement stack following the April 2026 bridge exploit. The review covered the MMR, Merkle multi-proof, and Patricia trie verifiers, as well as the IntentGatewayV2 settlement contract. We identified 14 issues (1 critical, 3 high, 5 medium, 4 low, 1 informational), most of them remediated and closed.

The critical finding echoes the original exploit: a duplicate-leaf-index bug in MerkleMultiProof lets an attacker with a single validator key fake a supermajority and seize control of the bridge. The same class of defect (silently ignored inputs and missing post-computation invariants) recurred across all three proof libraries and the escrow logic, pointing to systemic gaps rather than isolated bugs.

The most consequential findings affected proof verification integrity: defects in the Merkle multi-proof, Patricia trie, and MMR libraries could enable bridge takeover, key aliasing, and proof forgery. A second cluster of escrow accounting mismatches in the intent settlement layer enabled fund drainage, theft of orphaned balances, and permanent token locking.

Recommended path forward: Adopt invariant-first development for all trust-boundary code; document required correctness properties before implementation, encode them as on-chain assertions, and review each Solidity change against the corresponding Rust reference before merging. Implement defense-in-depth with recipient-side validation for cross-chain operations, establish adversarial test coverage with differential fuzzing against reference implementations, and deploy off-chain monitoring for anomalous bridge activity.

Content

1	Disclaimer	3
2	Motivation and scope	4
3	Findings summary	4
3.1	Detailed findings	6
3.2	S4-4: Duplicate leaf indices bypass Merkle multi-proof verification ...	6
3.3	S3-2: Same-chain partial fills over-release repeated input escrow	9
3.4	S3-3: Ethereum MPT verifier rejects inline child references	11
3.5	S3-6: Odd Ethereum leaf handling can alias different keys	13
3.6	S2-5: Empty trie handling compares raw nodes to empty-root hash	15
3.7	S2-12: Fee-on-transfer tokens can over-credit escrow accounting	17
3.8	S2-13: Predispatch flow harvests pre-existing balances	19
3.9	S2-14: Excess user native tokens become stuck in the gateway	21
3.10	S2-15: Same-chain partial fills prematurely finalize output legs	23
3.11	S1-7: MerklePatricia conflates absence with empty values	25
3.12	S1-9: Updating params can permanently brick gateway	27
3.13	S1-11: Universal MMR root forgery via empty leaves	29
3.14	S1-16: Single-leaf shortcut returns leaf hash without proof	32
3.15	S0-10: Protocol fee rounds down to zero on small inputs	34
4	Evolution suggestions	36
4.1	Technical recommendations	36
4.2	Procedural recommendations	37

1 Disclaimer

This report describes the findings and core conclusions derived from the audit carried out by Security Research Labs within the agreed-on timeframe and scope as detailed in Chapter 2. Please note that this report does not guarantee that all existing security vulnerabilities were discovered in the codebase exhaustively and that following all evolution suggestions described in Chapter 4 may not ensure all future code to be bug free.

2 Motivation and scope

Following the exploitation of Hyperbridge in April 2026, in which a missing invariant in the Solidity Merkle Mountain Range verifier allowed a forged proof to pass verification and authorize unauthorized cross-chain token operations, Polytope Labs engaged SRLabs to confirm the adequacy of the applied fix and assess the broader verification and settlement stack for similar or related vulnerabilities.

The original exploit leveraged a single gap in the MMR proof verification logic: the Solidity implementation did not check that all supplied leaves were consumed during peak decomposition, allowing an attacker to submit a forged leaf that silently bypassed verification. The attacker combined this with a zero challenge period configuration to immediately act on the forged state commitment, minting unauthorized Hyperbridge-wrapped DOT on Ethereum. The Rust reference implementation of the same algorithm was not affected, as it already enforced the invariant.

The scope of this engagement extends beyond verifying the patch. The incident demonstrated that a single verification gap in a bridge contract can be sufficient to authorize unauthorized cross-chain token operations, underscoring the need for a comprehensive review of the entire verification logic to ensure no similar gaps exist in adjacent code paths. Accordingly, the audit covers the following components:

Verification libraries (polytope-labs/solidity-merkle-trees):

- *MerkleMountainRange.sol* — MMR proof verification, including revalidation of the applied fix against the Rust reference implementation
- *MerkleMultiProof.sol* — Merkle multi-proof verification, used for BEEFY authority set membership and parachain header inclusion
- *MerklePatricia.sol* — Merkle Patricia trie verification (Substrate and Ethereum variants)

Intent settlement (hyperbridge/evm/src/apps/):

- *IntentGatewayV2.sol* — Order placement, fill, and cancellation entry points

The audit was executed within 3 weeks and was performed against commit 4580218ffca38c5e96ad13cd28b1949c33eeaf4 of the hyperbridge repository and against commit 135f2251c1e10677384f76a3514d0a6230bd8a56 in the solidity-merkle-trees dependency. The review combines manual source-level analysis guided by a threat model.

3 Findings summary

The audit identified 14 vulnerabilities, with most findings mitigated. Findings fall into two domains: proof verification integrity (7 issues, including the critical and all three high-severity findings) and intent settlement accounting (7 issues).

The most severe verification defects allow bridge takeover and proof forgery; the settlement defects enable fund drainage and locking. All are detailed below in order of severity.

Critical	1	
High	3	
Medium	5	
Low	4	
Informational	1	
Total Issues	14	

The below table shows an overview of all identified vulnerabilities, with their current mitigation status.

ID	Issue	Severity	Status
S4-4	Duplicate leaf indices bypass Merkle multi-proof verification	Critical	Mitigated
S3-2	Same-chain partial fills over-release repeated input escrow	High	Mitigated
S3-3	Ethereum MPT verifier rejects inline child references	High	Mitigated
S3-6	Odd Ethereum leaf handling can alias different keys	High	Mitigated
S2-5	Empty trie handling compares raw nodes to empty-root hash	Medium	Mitigated
S2-12	Fee-on-transfer tokens can over-credit escrow accounting	Medium	Mitigated
S2-13	Predispatch flow harvests pre-existing balances	Medium	Risk accepted
S2-14	Excess user native tokens become stuck in the gateway	Medium	Mitigated
S2-15	Same-chain partial fills prematurely finalize output legs	Medium	Mitigated
S1-7	MerklePatricia conflates absence with empty values	Low	Mitigated
S1-9	Updating params can permanently brick gateway	Low	Mitigated

S1-11	Universal MMR root forgery via empty leaves	Low	Mitigated
S1-16	Single-leaf shortcut returns leaf hash without proof	Low	Mitigated
S0-10	Protocol fee rounds down to zero on small inputs	Info	Risk accepted

Table 1: Findings overview

3.1 Detailed findings

3.2 S4-4: Duplicate leaf indices bypass Merkle multi-proof verification

Attack scenario	An attacker duplicates one authority vote to fake a supermajority
Classification	Insecure Business Logic
Component	MerkleMultiProof, BeefyV1
Tracking	https://github.com/seunlanlege/hyperbridge/issues/4
Attack impact	Bridge takeover / arbitrary token minting
Severity	Critical
Status	Mitigated

Background

MerkleMultiProof is a Solidity library for verifying inclusion of multiple leaves in a Merkle tree using a compact multi-proof. **BeefyV1** uses it to confirm that a supermajority of the relay chain's authority set has signed a commitment before accepting a new MMR root.

Issue description

The library bug.

CalculateRoot accepts a caller-supplied **leaves** array and validates each entry with a single range check:

```
if (leaves[i].index >= leafCount) revert LeafIndexOutOfBounds();
```

There is no check that indices are distinct or ordered. A caller may therefore supply the same leaf index multiple times, and all copies pass validation. Each duplicate is assigned the same position in the tree and enters **_walk** as a separate entry.

_walk processes entries left to right at each tree level. When two adjacent entries share a sibling, i.e. their positions differ by exactly one and form a left/right pair, it merges them into their parent using only those two hashes. When no such sibling is present in the set, it consumes the next hash from the caller-supplied **proof** array instead. Because two duplicate entries sit at the same position, they never form a sibling pair. Each one therefore independently reaches into **proof** and fetches its

own sibling hash, producing the same parent hash independently. This repeats level by level. The outer loop in `_walk` terminates as soon as the first entry in the working set reaches the root position, returning `hashes[0]` unconditionally, it does not check whether other entries also reached the root, nor whether the final working set has been reduced to a single element.

The consequence is that N copies of the same leaf, paired with N copies of the same sibling proof path, cause `_walk` to compute the correct root N times in parallel and then return it. `VerifyProof` compares that result against the trusted root and returns `true`. The verifier has accepted N claimed leaf memberships while only one real Merkle path was checked.

To make this concrete: consider a two-leaf tree containing leaves `A` at index 0 and `B` at index 1, whose root is `H(A, B)`. A prover submitting `leaves = [(0, A), (0, A)]` with `proof = [B, B]` causes `_walk` to compute `H(A, B)` for each duplicate entry independently, both entries reach the root position, the loop exits, and `VerifyProof` returns `true`. No membership of a second leaf was proven.

The downstream impact in BeefyV1.

`verifyMmrUpdateProof` verifies that enough members of the relay chain's authority set have signed a given commitment. It measures participation by counting the entries in the caller-supplied `votes` array:

```
uint256 signatures_length =
relayProof.signedCommitment.votes.length;
if (!checkParticipationThreshold(signatures_length,
trustedState.currentAuthoritySet.len) ...) {
    revert SuperMajorityRequired();
}
```

It then builds one Merkle leaf per vote, recovering the signer's address from the ECDSA signature and pairing it with the vote's claimed authority index:

```
for (uint256 i = 0; i < signatures_length; i++) {
    Vote memory vote = relayProof.signedCommitment.votes[i];
    address authority = ECDSA.recover(commitment_hash,
vote.signature);
    authorities[i] = MerkleMultiProof.Leaf(
        vote.authorityIndex,
        keccak256(abi.encodePacked(authority))
    );
}
```

Neither the threshold check nor the leaf-building loop enforces that `vote.authorityIndex` is unique across votes. An attacker who controls one valid authority key at index k can therefore submit any number of votes all carrying `authorityIndex = k` and the same signature. Each vote passes ECDSA recovery, the signature is genuine and produces an identical Merkle leaf. The threshold check sees as many votes as the attacker chooses to include and passes once that count reaches the required supermajority. The subsequent call to `VerifyProof` accepts the duplicated leaf set via the library bug described above. From the contract's perspective, the commitment appears to have been signed by a supermajority of distinct authorities.

Risk

An attacker with access to one authority key, achievable by running a single Polkadot validator or by compromising one, can unilaterally satisfy the supermajority check and submit a forged MMR root. From there the attacker can rotate the trusted authority set to one they fully control, after which the bridge is permanently compromised. The attack requires no cryptographic break and no special on-chain permissions beyond submitting a crafted transaction. The **BeefyV1FiatShamir** path is not affected because it binds each vote to an externally determined challenged index, preventing index reuse.

Mitigation

CalculateRoot should enforce that leaf indices are strictly increasing before entering **_walk**:

```
for (uint256 i = 1; i < len; ) {
    if (leaves[i].index <= leaves[i - 1].index) revert
    LeafIndexNotSorted();
    unchecked { ++i; }
}
```

Strict monotonicity rejects both duplicates and out-of-order inputs without additional data structures. As defence in depth, **verifyMmrUpdateProof** in **BeefyV1** should also assert that **vote.authorityIndex** values are strictly increasing across the votes array before building the leaf set, so that the call site remains safe against any future library regression.

Note: Polytope has informed SRLabs that this issue was independently identified by their team and patched in a separate branch prior to the audit. This branch was not part of the audit scope and the remediation was not communicated to SRLabs prior to the engagement. The finding accurately reflects the state of the code reviewed. SRLabs has verified that the fix in the branch addresses the root cause of the issue.

3.3 S3-2: Same-chain partial fills over-release repeated input escrow

Attack scenario	An attacker fills one leg and drains escrow
Classification	Insecure Business Logic
Component	IntentGatewayV2 same-chain fills
Tracking	https://github.com/seunlanlege/hyperbridge/issues/2
Attack impact	Direct fund theft via escrow over-release
Severity	High
Status	Mitigated

Background

`IntentGatewayV2` is the protocol component that manages escrow-backed orders between users and solvers. It supports both orders settled on one blockchain and orders settled across different blockchains; this issue affects the one-blockchain, same-chain settlement mode.

Issue description

`order.inputs` is the list of asset entries the user deposits into escrow, and `order.output.assets` is the list of asset entries the user wants the solver to deliver in return. For orders whose escrow and fulfillment both happen on the same blockchain, `fillOrder` requires these two lists to have the same number of entries, and `_fillSameChain` then processes them by index. This means the contract treats the order as a list of per-entry exchanges rather than a single aggregated basket. At the same time, escrow is stored only by `(commitment, token)` in `_orders`, not by input index.

When an order is placed, repeated input-token entries are merged into the same escrow bucket in `placeOrder`:

```
for (uint256 i; i < inputsLen;) {
    if (order.inputs[i].amount == 0) revert InvalidInput();
    address token =
    address(uint160(uint256(order.inputs[i].token)));
    ...
    _orders[commitment][token] += reducedInputs[i].amount;
}
```

Later, if one output entry becomes fully satisfied, `_fillSameChain` releases escrow by reading the full bucket for `order.inputs[i].token` in `_fillSameChain`:

```
uint256 escrowedAmount;
if (amountFilled == totalRequired) {
    escrowedAmount =
    _orders[commitment][address(uint160(uint256(order.inputs[i].token)
))]];
} else {
    escrowedAmount = (order.inputs[i].amount * fillAmount) /
    totalRequired;
}
```

Consider a same-chain order with:

```
inputs:
  [0] USDC 1200
  [1] USDC 1000

outputs:
  [0] WETH 0.5
  [1] DAI 1000
```

This is a natural per-entry representation: both entries spend USDC, but they ask for different outputs. When the order is placed, both USDC inputs are merged into `_orders[commitment][USDC] = 2200`.

If a solver fills only the first pair by delivering `WETH 0.5`, the solver should receive only the `USDC 1200` attached to that pair. Instead, because both USDC inputs were merged into a single escrow bucket when the order was placed, completing the first pair unlocks the full `2200 USDC` bucket. `_withdraw` then transfers all `2200 USDC` to the solver, even though the second pair (`USDC 1000 -> DAI 1000`) has not been filled. The order remains open, but the escrow backing the second pair is already gone.

Risk

This causes direct loss for users on orders whose escrow and fulfillment both happen on the same blockchain. If an order repeats the same input token across multiple entries, a solver can complete one pair and receive the full combined escrow for that token. The user loses the excess amount, and any remaining pair can stay open without the assets that were meant to back it.

This is also not a contrived order shape. In the current codebase, a same-chain order is only fillable when the input and output lists have the same number of entries. As a result, a user who wants to sell one token type for multiple different outputs must split that input across multiple entries to produce a fillable order. Repeated input-token entries are therefore a natural order pattern under the current interface, which makes the over-release practically relevant.

Mitigation

Use one accounting model consistently. The preferred fix is to store and release escrow per entry rather than per token. If the design intends token-level aggregation, then escrow release should also be computed at the token level instead of by array index. At minimum, reject orders that repeat an input token until the accounting is made consistent.

3.4 S3-3: Ethereum MPT verifier rejects inline child references

Attack scenario	An attacker does present valid proofs as missing
Classification	Insecure Business Logic
Component	MerklePatricia, EthereumTrieDB
Tracking	https://github.com/seunlanlege/hyperbridge/issues/3
Attack impact	Cross-chain proof verification denial
Severity	High
Status	Mitigated

Background

MerklePatricia is the Solidity component that verifies Substrate and Ethereum trie proofs on-chain. Its **VerifyEthereumProof** path decodes Ethereum trie nodes and walks the proof from the trusted root to the queried key.

Issue description

Ethereum's Merkle Patricia Trie encoding allows a parent to reference a child either by hash or by embedding the child node inline when the encoded child is shorter than 32 bytes. The [official Ethereum trie documentation](#) states:

When one node is referenced inside another node, what is included is `keccak256(rlp.encode(node))`, if `len(rlp.encode(node)) >= 32` else `node`.

The library's generic trie layer already models both cases through **NodeHandle** and **TrieDB.load**, but the Ethereum decoder does not preserve that distinction.

In **decodeExtension()**, the child reference is always converted into a hash handle. In **decodeBranch()**, a child is treated as present only when the decoded entry is exactly 32 bytes long. The relevant logic is:

```
if (dataAsBytes.length != 32) {
    childrens[i] = NodeHandleOption(
        false,
        NodeHandle(false, bytes32(0), false, new bytes(0))
    );
} else {
    bytes32 data = Bytes.toBytes32(dataAsBytes);
    childrens[i] = NodeHandleOption(
        true,
        NodeHandle(true, data, false, new bytes(0))
    );
}
```

As a result, valid inline branch children are misclassified as absent, and valid inline extension children cannot be followed as inline nodes. **VerifyEthereumProof** later consumes these decoded nodes while traversing the proof. Once **decodeBranch()** has marked an inline child as absent, the lookup follows that result and stops without assigning a value. Once **decodeExtension()** has forced an inline child into a hash handle, the verifier cannot process the proof in the form emitted by Ethereum clients.

This makes the verifier disagree with canonical Ethereum trie semantics. A proof that is valid under the standard encoding can therefore be rejected or interpreted as non-membership by this implementation.

Risk

Any integration that relies on this library to verify Ethereum state, storage, receipt, or transaction proofs can disagree with real Ethereum clients on whether a proof is valid. In practice, this can cause valid membership proofs to fail or to be misread as missing data, which undermines higher-level logic built on top of the verifier.

Mitigation

Update the Ethereum decoder to preserve both legal child reference forms. `decodeBranch()` and `decodeExtension()` should emit hash handles for 32-byte references and inline handles for shorter embedded child nodes, then let `TrieDB.load()` resolve either form through the existing generic abstraction.

3.5 S3-6: Odd Ethereum leaf handling can alias different keys

Attack scenario	An attacker submits a valid Ethereum proof for key K that the verifier accepts for a different key K'
Classification	Insecure Business Logic
Component	MerklePatricia, EthereumTrieDB
Tracking	https://github.com/seunlanlege/hyperbridge/issues/6
Attack impact	Proof integrity bypass / state spoofing
Severity	High
Status	Mitigated

Background

MerklePatricia is the Solidity component that verifies Ethereum and Substrate trie proofs on-chain. For Ethereum proofs, **VerifyEthereumProof** decodes compact-encoded trie paths and compares the decoded leaf suffix against the remaining queried key.

Issue description

Ethereum's compact trie encoding distinguishes odd and even remaining path lengths for 2-item nodes. The [official Ethereum trie documentation](#) states:

The flagging of both odd vs. even remaining partial path length and leaf vs. extension node as described above reside in the first nibble of the partial path of any 2-item node.

For even remaining path length (0 or 2), another 0 "padding" nibble will always follow.

This rule applies to both **leaf** and **extension** nodes. In the odd case, the first byte carries both flags and the first real path nibble. However, **decodeLeaf()** always strips the first full byte and rebuilds the leaf key with offset 0:

```
bytes memory data = decoded[1].toBytes();
// Remove the first byte, which is the prefix and not present in
// the user provided key
leaf.key = NibbleSlice(Bytes.substr(decoded[0].toBytes(), 1), 0);
leaf.value = NodeHandle(false, bytes32(0), true, data);
```

By contrast, **decodeExtension()** derives **isOdd** and preserves the nibble offset. This is the first contributing defect: odd leaf paths lose one real nibble during decoding.

VerifyEthereumProof() then introduces a second contributing defect when it realigns the remaining queried key at the leaf before comparing it:

```
uint256 offset = keyNibbles.offset % 2 == 0
    ? keyNibbles.offset / 2
    : keyNibbles.offset / 2 + 1;
keyNibbles = NibbleSlice(
    NibbleSliceOps.bytesSlice(keyNibbles.data, offset),
    0
);
```

When the queried key is at an odd nibble offset, this rounds up to the next byte boundary and drops the queried nibble at that position as well.

These two defects cancel for the genuine key, but they also make different keys compare equal. In the odd-leaf case, `decodeLeaf()` discards the first real leaf nibble from the proof, and `VerifyEthereumProof()` discards the corresponding queried nibble from the key before running `NibbleSliceOps.eq(leaf.key, keyNibbles)`. As a result, a valid proof for one key can also be accepted for a different queried key that shares the same traversed branch prefix and the same remaining leaf-suffix nibbles after the first, differing only at the dropped first odd-suffix nibble.

Risk

This is an integrity failure in the Ethereum proof verifier. A valid proof for key `K` can be accepted as a proof for a different key `K'`, causing the verifier to return the proven value for the wrong queried key.

Any integration that relies on `VerifyEthereumProof` to bind a returned value to a specific Ethereum state or storage key can therefore be tricked into accepting incorrect key/value relationships. This breaks the core security guarantee of the proof verifier.

Mitigation

Fix both sides of the comparison.

`decodeLeaf()` should be parity-aware in the same way `decodeExtension()` already is: it should derive odd versus even from the compact-path flags and preserve the embedded first path nibble in the odd case instead of always removing one full byte. `VerifyEthereumProof()` should also preserve nibble alignment when comparing the remaining queried key at a leaf, rather than rounding odd offsets up to the next byte boundary.

3.6 S2-5: Empty trie handling compares raw nodes to empty-root hash

Attack scenario	An attacker does trigger empty-trie proof reverts
Classification	Insecure Business Logic
Component	MerklePatricia, EthereumTrieDB
Tracking	https://github.com/seunlanlege/hyperbridge/issues/5
Attack impact	Proof verification denial (empty tries)
Severity	Medium
Status	Mitigated

Background

MerklePatricia is the Solidity component that verifies Ethereum and Substrate trie proofs on-chain. For Ethereum proofs, **VerifyEthereumProof** loads the root node from the supplied proof and decodes it as an Ethereum trie node before traversing the key path.

Issue description

VerifyEthereumProof builds each proof entry as `(keccak256(raw_node), raw_node)` and then starts verification by looking up the raw node bytes for `root` and passing those bytes into the Ethereum decoder. The empty-trie special case inside `decodeNodeKind()` compares those raw bytes directly against `HASHED_NULL_NODE`:

```
bytes constant HASHED_NULL_NODE =
    hex"56e81f171bcc55a6ff8345e692c0f86e5b48e01b996cad001622fb5e363b4
    21";

if (Bytes.equals(encoded, HASHED_NULL_NODE)) {
    node.isEmpty = true;
    return node;
}
```

This mixes up two different representations: `encoded` is the raw RLP-encoded trie node, while `HASHED_NULL_NODE` is the hash of the RLP-encoded empty trie node. In other words, the code compares a preimage to a hash. That equality check is wrong on first principles: a correct check must compare preimage-to-preimage or hash-to-hash.

Because the empty-trie special case is checked against the wrong representation, an empty Ethereum trie is not handled as a normal empty/non-membership case. When the special case is missed in `decodeNodeKind()`, the function falls through into ordinary RLP list decoding and reverts at `RLPReader.toList()`, because the empty-node encoding is not an RLP list. The proof is therefore rejected as an exceptional error instead of cleanly returning an empty result.

Risk

Valid non-membership proofs against empty Ethereum tries are rejected as exceptional failures rather than being processed as normal empty proofs. Integrations that rely on this verifier therefore fail on empty-storage cases instead of receiving a deterministic non-membership result. This is a practical case for

Ethereum storage proofs because externally owned accounts and contracts that have never written storage use the canonical empty storage trie.

Mitigation

Handle empty tries using the correct representation. If the check is performed on **root**, compare hash-to-hash against **HASHED_NULL_NODE**. If the check is performed on **encoded**, compare preimage-to-preimage against the canonical raw empty-node encoding.

3.7 S2-12: Fee-on-transfer tokens can over-credit escrow accounting

Attack scenario	A fee-on-transfer token over-credits escrow, allowing withdrawal of pooled funds
Classification	Insecure Business Logic
Component	IntentGatewayV2 same-chain fills
Tracking	https://github.com/seunlanlege/hyperbridge/issues/12
Attack impact	Escrow accounting corruption / fund drainage
Severity	Medium
Status	Mitigated

Background

In the direct-escrow path of `placeOrder`, the gateway transfers the user-specified input amount and then credits escrow using the requested post-protocol-fee amount, not the amount actually received:

```
IERC20(token).safeTransferFrom(msg.sender, address(this),
order.inputs[i].amount);
_orders[commitment][token] += reducedInputs[i].amount;
```

Issue description

For standard ERC-20s, the requested amount and the gateway's balance delta are equal. For fee-on-transfer or deflationary tokens, they can differ.

Example with `protocolFeeBps == 0` and a token that burns 1% on transfer:

1. User places an order with `input.amount = 100`.
2. The gateway receives only `99`.
3. `_orders[commitment][token]` is credited with `100`.

The withdrawal path later trusts `_orders` and attempts to transfer the recorded amount:

```
uint256 escrowed = _orders[body.commitment][token];
_orders[body.commitment][token] = escrowed - amount;
IERC20(token).safeTransfer(beneficiary, amount);
```

If the gateway only holds this commitment's balance, withdrawal reverts due to insufficient tokens. If other commitments use the same token, an earlier withdrawal can succeed by consuming pooled token balance that economically belongs to later orders.

The same issue applies to cross-chain orders at source-chain redemption time, because redemption uses the same `_withdraw` ledger.

Risk

Affected orders can become unwithdrawable, or one withdrawal can consume same-token liquidity from another commitment. The contract accepts arbitrary ERC-20

input tokens, therefore accounting correctness cannot be guaranteed for non-standard transfer implementations.

Mitigation

If fee-on-transfer tokens must be supported, compute escrow credit from the actual received amount instead of the user-supplied amount and compute the protocol fee from that received amount. Otherwise only allow known standard ERC-20s as input tokens or verify during placement that the gateway's balance equals the requested amount.

3.8 S2-13: Predispatch flow harvests pre-existing balances

Attack scenario	An attacker harvests orphaned token balances from the shared CallDispatcher
Classification	Insecure Business Logic
Component	IntentGatewayV2 same-chain fills
Tracking	https://github.com/seunlanlege/hyperbridge/issues/13
Attack impact	Direct fund theft via orphaned balances
Severity	Medium
Status	Risk accepted

Background

`placeOrder` supports a "predispatch" mode where the user provides arbitrary calldata that the gateway's `CallDispatcher` runs on their behalf - e.g. unwrap an LP token, swap, or aggregate funds - before the resulting tokens are pulled into escrow. The flow at `IntentGatewayV2.sol#L194-L251` deposits `predispatch.assets` to the dispatcher, runs `predispatch.call`, then sweeps the dispatcher for each declared input token.

Issue description

The `CallDispatcher` is *stateless and shared*: every `IntentGateway` user routes through the same dispatcher contract, and `dispatch` does not sweep residual balances at the end of a call. Any token left on the dispatcher after one user's predispatch persists until another caller pulls it.

The predispatch sweep at `IntentGatewayV2.sol#L221-L249` reads the entire balance of each input token on the dispatcher and transfers that whole balance to the gateway, then credits only `reducedInputs[i].amount` to the caller's escrow:

```
balance = IERC20(token).balanceOf(dispatcher);
if (balance < requiredAmount) revert InvalidInput();
transferCalls[i] = Call({
    to: token,
    value: 0,
    data: abi.encodeWithSelector(IERC20.transfer.selector,
address(this), balance)
});
...
uint256 dust = balance - requiredAmount;
if (dust > 0) emit DustCollected(token, dust);
_orders[commitment][token] += reducedInputs[i].amount;
```

The variable `dust` and the `DustCollected` event name are misleading: this value is not bounded to small "dust" amounts. It is the full leftover balance of the input token on the shared dispatcher at the moment of the sweep. Whatever an attacker (or a sequence of unrelated honest users) has parked there will be picked up in one go. For an LP-redemption predispatch returning a side token in the same order of magnitude as the primary asset, the residual is the same order of magnitude as that order's notional, not a dust amount.

Consider the following scenario:

1. Victim holds 50 USDT.
2. Victim places an order whose predispatch deposits the 50 USDT on the dispatcher; the sweep skips USDT because it is not in `inputs`. Victim now has 0 USDT, the dispatcher holds 50 USDT, and `_orders[victimCommitment][USDT] == 0`.
3. Attacker places an order with `inputs = [{USDT, 50}]` and an empty `predispatch.call`. The sweep reads `balanceOf(dispatcher, USDT) == 50`, transfers all 50 USDT to the gateway, and credits the attacker's commitment escrow.
4. Attacker calls `cancelOrder`; `_withdraw` releases the 50 USDT to the attacker's wallet.

The 50 USDT moved from the victim's wallet, through the orphaned dispatcher balance, into the attacker's wallet. The victim cannot recover it: `_orders[victimCommitment][USDT] == 0` because USDT was never in their `inputs`, so cancellation has nothing to refund.

Risk

The funds an attacker can claim are bounded by what honest users have left on the shared dispatcher, not by any "dust" floor. A single predispatch that produces secondary tokens (e.g LP token redemption) that the user did not declare in `inputs`, can strand a balance comparable to the order's primary notional. This is not a small remainder of dust.

Multiple users' leftovers also accumulate on the same dispatcher between harvests, so the harvestable balance grows monotonically until someone takes it. Once funds are present, they can be arbitrarily claimed.

Exploitation requires an attacker to monitor `balanceOf(dispatcher, _)` for non-zero balances of any plausible token, then submit a `placeOrder` with the right predispatch shape. The predispatch.assets minimum is `1 wei` of any token (or native ETH), so the entry cost is negligible.

Mitigation

We recommend computing the delta of dispatcher balances across the user's own predispatch rather than taking the full balance. Snapshot before, snapshot after, sweep only (`after - before`). This eliminates the harvest both for ERC-20 and native, regardless of how dirty the shared dispatcher gets.

The `duplicate input tokens issues` in the predispatch path interacts with this fix. They should be remediated together.

3.9 S2-14: Excess user native tokens become stuck in the gateway

Attack scenario	A user overpays native ETH during order placement and cannot recover the excess
Classification	Insecure Business Logic
Component	IntentGatewayV2 same-chain fills
Tracking	https://github.com/seunlanlege/hyperbridge/issues/14
Attack impact	Permanent user fund locking
Severity	Medium
Status	Mitigated

Background

`IntentGatewayV2` accepts native ETH (or chain-native) to fund three things during `placeOrder`: predispatch asset transfers (`IntentGatewayV2.sol#L194-L216`), direct-escrow inputs for native-token legs (`L256-L259`), and the Hyperbridge relayer-fee payment, including an optional `swapETHForExactTokens` path when `msgValue > 0` (`L271-L284`). The function tracks remaining native via the local `msgValue` counter and decrements it at each consumption point.

Issue description

The local counter `msgValue` is never reconciled against `address(this).balance` and there is no refund of unspent ETH at the end of `placeOrder`. Two leakage paths exist:

1. **Direct-escrow / predispatch path** - the user might send more `msg.value` than is actually consumed (UI rounding, over-budgeting, or simply a fixed-size `msg.value` for an order whose native input was less). After the inputs loop exits, any remaining `msgValue` is silently retained by the contract because nothing reads it again. There is no `if (msgValue > 0) refund` clause.
2. **Fee-swap path** - when `msgValue > 0` and `order.fees > 0`, the contract calls `swapETHForExactTokens{value: msgValue}` (`L278-L280`):

```
IUniswapV2Router02(uniswapV2).swapETHForExactTokens{value:
msgValue}(
    order.fees, path, address(this), block.timestamp
);
```

The router uses the exact amount needed and refunds the unused ETH back to `msg.sender` of the swap call - which is the gateway, not the user. The gateway has a `receive()` so it accepts the refund silently. Combined with the absence of a refund step, the user's overpayment-or-slippage-buffer becomes protocol-controlled balance.

The order's escrow ledger only records what was *intended* to be escrowed (`order.inputs[i].amount` for native legs and `order.fees` for the fee bucket); the trapped surplus is not associated with the commitment, so cancellation cannot recover it. The same shape exists for `fillOrder` native overpayment in `IntrinsicIntents.sol#L99-L102` and `ExtrinsicIntents.sol#L120-L124`: solver overpayment is not refunded.

The trapped balance is technically reachable by Hyperbridge governance via the **SweepDust** admin path, but only as protocol revenue - never to the user who paid it.

Risk

This results in financial loss per affected user, scaling with the size of over-payment. This is a self-inflicted loss caused by missing refund logic, and not exploitable by an attacker.

Over-payment happens routinely in practice. UIs that pre-bundle calldata for offline signing may round **msg.value** up; integrations that share **msg.value** across multiple sub-actions over-budget; users who set a slippage buffer for the fee-swap leg lose that buffer permanently.

Mitigation

We recommend adding an explicit refund of any remaining **msgValue** at the end of the relevant paths to avoid users losing native tokens due to over-payment. Alongside this implement re-entrancy protection as safe-guard to guard against this refund call. Alternatively, residue native tokens may be recorded per user and a pull-based refund model can be implemented.

3.10 S2-15: Same-chain partial fills prematurely finalize output legs

Attack scenario	A solver omits duplicate output legs while the order finalizes as fully filled
Classification	Insecure Business Logic
Component	IntentGatewayV2 same-chain fills
Tracking	https://github.com/seunlanlege/hyperbridge/issues/15
Attack impact	Partial fund theft via premature finalization
Severity	Medium
Status	Mitigated

Background

This is the output-side extension of the duplicate-input-token issue. Same-chain fills are modeled as per-index exchanges: `order.inputs[i]` backs `order.output.assets[i]`. However, the implementation tracks both sides in token-keyed buckets rather than leg-keyed buckets.

The duplicate-input issue comes from `_orders[commitment][inputToken]`. This issue comes from the symmetric output-side bucket, `_partialFills[commitment][outputToken]`.

Issue description

In `_fillSameChain`, fill progress is keyed only by output token:

```
uint256 alreadyFilled = _partialFills[commitment][outputToken];
uint256 remaining = totalRequired - alreadyFilled;
```

If the same output token appears in multiple legs, filling one leg updates the shared bucket for every leg with that token (`IntrinsicIntents.sol#L94-L95`). A later duplicate leg can then enter the early-skip branch as already complete (`IntrinsicIntents.sol#L73-L76`):

```
if (remaining == 0 || solverAmount == 0) {
    if (solverAmount == 0 && remaining > 0) isFullyFilled = false;
    continue;
}
```

Because `remaining == 0` does not set `isFullyFilled = false`, the order can finalize even though that specific output leg was never paid.

Unlike the duplicate-input-token issue, this does not require duplicate input tokens. Distinct input tokens are enough; the bug is caused by duplicate output tokens sharing one fill-progress bucket.

Risk

A solver can omit one or more duplicate output legs while the order is marked fully filled. The skipped input legs remain stranded because cancellation is blocked after finalization. Duplicate output tokens are a plausible order shape, especially when

users split one desired asset across multiple input legs. The contract neither rejects nor correctly supports this shape.

Mitigation

Apply the same fix class as the duplicate-input-token issue: either reject duplicate output tokens during order creation/fill validation, or track fill progress per leg, e.g. by `(commitment, index)` instead of `(commitment, outputToken)`.

The smallest patch is to reject duplicate `order.output.assets[i].token` values alongside the duplicate-input-token rejection.

Note: This issue can be considered a counterpart (or extension) of [Same-chain partial fills over-release repeated input escrow](#). These issues should be resolved together.

3.11 S1-7: MerklePatricia conflates absence with empty values

Attack scenario	An attacker submits empty-valued proofs causing misclassification
Classification	Insecure Business Logic
Component	MerklePatricia, VerifySubstrateProof, SubstrateTrieDB
Tracking	https://github.com/seunlanlege/hyperbridge/issues/7
Attack impact	State misinterpretation at API boundary
Severity	Low
Status	Mitigated

Background

MerklePatricia verifies Merkle Patricia trie proofs for Substrate and Ethereum. Its `VerifySubstrateProof()` entrypoint returns the values associated with queried keys under a trusted Substrate trie root.

Issue description

The `StorageValue` result type contains only `key` and `value`, and the current implementation represents non-membership as `value = ""`. This follows directly from the verifier flow in `VerifySubstrateProof()`: `values[i].value` is assigned only when traversal reaches a matching leaf or branch value. If traversal stops at a mismatch, missing child, or empty node, the function returns the default empty bytes instead.

That encoding is ambiguous, because empty bytes are also a legitimate stored value. `SubstrateTrieDB.decodeLeaf()` can decode an existing leaf value as `bytes("")`: if the inline value length is `0`, the key is still present, but its value is empty bytes:

```
uint256 len = ScaleCodec.decodeUIntCompact(input);
handle.isInline = true;
handle.inLine = Bytes.read(input, len);
```

Substrate does not provide a first-class FRAME storage set primitive, so set-like state is commonly represented with a map whose value is `()`. For example, a pallet may store `ProcessedMessages[msg_id] = ()` to record that `msg_id` was already processed. Since `()` SCALE-encodes to empty bytes, a valid membership proof for `ProcessedMessages[msg_id]` returns the same `value = ""` as non-membership under the current API, so a caller that treats empty bytes as “not found” would conclude incorrectly that the message was never processed.

Risk

Callers that interpret `value.length == 0` as non-membership can make incorrect decisions, because valid membership proofs for empty values are indistinguishable from absence. This is primarily an API correctness issue, but it can become security-relevant when proof presence gates authorization, replay protection, or state-transition checks.

Mitigation

Return proof results in a form that distinguishes non-membership from membership with an empty value. For example, the verifier can return presence explicitly alongside the value or use any other representation that makes those two states unambiguous without relying on `value.length` alone. If `StorageValue` is intended to remain shared across both verifier entrypoints, it should distinguish absence from empty values uniformly for both the Substrate and Ethereum proof paths.

3.12 S1-9: Updating params can permanently brick gateway

Attack scenario	A malformed governance update permanently bricks the gateway
Classification	Insecure Business Logic
Component	IntentGatewayV2 same-chain fills
Tracking	https://github.com/seunlanlege/hyperbridge/issues/9
Attack impact	Permanent gateway denial of service
Severity	Low
Status	Mitigated

Background

IntentGatewayV2 stores operational configuration in `Params`: `host`, `dispatcher`, `solverSelection`, `surplusShareBps`, `protocolFeeBps`, and `priceOracle`. Future changes are applied through Hyperbridge governance via `_updateParams`.

Issue description

`_updateParams` replaces the entire `params` struct and writes destination-specific protocol fees without validating any value:

```
emit ParamsUpdated({previous: _params, current: update.params});
_params = update.params;
...
_destinationProtocolFees[stateMachineId] = feeBps;
```

Malformed governance updates can therefore amplify an operational mistake or governance compromise into a broader outage:

- `host`: if set to `address(0)` or the wrong host, future `onlyHost` governance messages cannot repair the config.
- `dispatcher`: if set to `address(0)` or a non-dispatcher, `predispatch` and `output-call` orders revert.
- `surplusShareBps`: if greater than `10_000`, surplus splitting can underflow in same-chain and cross-chain fills (`IntrinsicIntents.sol#L87-L88`, `ExtrinsicIntents.sol#L114-L115`).
- `protocolFeeBps`: if greater than `10_000`, fee reduction underflows during order placement; if equal to `10_000`, reduced escrow becomes zero (`IntentGatewayV2.sol#L170-L179`).
- `priceOracle`: if set to a reverting or non-contract address, fills can be blocked when `recordSpread` is called.
- `destinationFeeBps`: each override is also unbounded and feeds the same protocol-fee calculation as `protocolFeeBps`.

Risk

A bad params update can freeze placement, fills, call execution, or future governance recovery. This is especially relevant during an ongoing governance incident, where bounding params limits the risk of malicious configurations. Updates are

governance-gated, but the contract currently provides no on-chain guardrails against malformed or adversarial values.

Mitigation

We recommend implementing defense-in-depth approach to configuration updates by validating all params before accepting updates, for example:

```
function _validateParams(Params memory p) internal view {
    if (p.host == address(0) || p.host.code.length == 0) revert
    InvalidInput();
    if (p.dispatcher == address(0) || p.dispatcher.code.length ==
    0) revert InvalidInput();
    if (p.surplusShareBps > 10_000) revert InvalidInput();
    if (p.protocolFeeBps >= 10_000) revert InvalidInput();
    if (p.priceOracle != address(0) && p.priceOracle.code.length
    == 0) revert InvalidInput();
}
```

Apply this in both `setParams` and `_updateParams`. For every `DestinationFee`, also require:

```
if (destinationFee.destinationFeeBps >= 10_000) revert
InvalidInput();
```

3.13 S1-11: Universal MMR root forgery via empty leaves

Attack scenario	An attacker does not prove any leaf at all, and instead submits <code>leaves = []</code> , a crafted proof, and a chosen <code>leafCount</code> so that the MMR library reconstructs the trusted root from attacker-supplied peak values.
Classification	Proof verification bypass
Component	EVM dependency / @polytope-labs/solidity-merkle-trees / MerkleMountainRange
Tracking	https://github.com/seunlanlege/hyperbridge/issues/11
Attack impact	Authentication / inclusion-check bypass
Severity	Low
Status	Mitigated

Background

`MerkleMountainRange` is the EVM-side inclusion verifier for Merkle Mountain Range proofs. Its intended behavior is to reconstruct the trusted MMR root from a set of claimed leaves, the sibling data in `proof`, and the total `leafCount`, and return success only when the supplied leaves are genuinely part of that MMR. To do that safely, it must fully decompose the MMR into peaks, fill every required peak, and bag those peaks into the final root without accepting partial reconstruction.

Issue description

`CalculateRoot` fails to enforce that peak reconstruction completed successfully. When it reaches a subtree that contains no supplied leaves, it tries to source that subtree's peak from `proof`. If the proof is already exhausted, it exits the decomposition loop with `break` instead of reverting (`MerkleMountainRange.sol#L136-L141`):

```
if (subtreeLeaves.length == 0) {
    if (proofIter.data.length == proofIter.offset) {
        break;
    } else {
        _push(peakRoots, _next(proofIter));
    }
}
```

After exiting the loop, the function checks only for leftover leaves (`MerkleMountainRange.sol#L152`):

```
if (leafIter.length != 0) revert OutOfBoundsLeaves();
```

It does **not** check that all required proof data was consumed or that all expected peaks were populated. This is the core bug. `peakRoots` is allocated to `popcount(leafCount)`, but the function later bags only the prefix that was actually written. That means the returned root may be derived from a partial peak set rather than from the full MMR decomposition.

The simplest forgery is the single-peak case, i.e. any `leafCount` that is a power of two. Then `popcount(leafCount) == 1`, so the MMR has exactly one peak. If the attacker passes:

```
Leaf[] memory leaves = new Leaf[](0);
bytes32[] memory proof = new bytes32[](1);
proof[0] = trustedRoot;

bool ok = MerkleMountainRange.VerifyProof(trustedRoot, proof,
leaves, 4);
assert(ok);
```

the function behaves as follows:

1. It sees one expected peak.
2. Because `leaves` is empty, `_subtreeLeaves` returns no leaves for that peak.
3. It reads `proof[0]` and stores it as the peak root.
4. There is only one peak, so the bagging loop is skipped.
5. The function returns `peakRoots[0]`, which is the attacker-supplied `trustedRoot`.

No leaf was proven and no membership path was validated. The verifier simply accepted an attacker-chosen peak as the final MMR root.

This is stronger than ordinary proof malleability. The attacker is not just adding junk to an otherwise valid proof. They can manufacture a successful verification result for an arbitrary trusted root whenever the calling context lets them control `(proof, leaves, leafCount)` and does not reject empty-leaf verification.

The same structural flaw affects multi-peak MMRs. For example, with `leafCount = 14`, the MMR decomposes into three peaks. If the attacker provides only two peak values, the function stops early and bags only those two peaks instead of reverting. The result is again a root computed from incomplete state.

Risk

This is a critical library-level verification bypass. In any integration where an attacker can influence `proof`, `leafCount`, and whether `leaves` may be empty, the verifier can degenerate into:

```
VerifyProof(trustedRoot, [trustedRoot], [], 1) == true
```

If that check gates a sensitive action, the attacker can satisfy the gate without proving inclusion of any real leaf. Depending on the integration, that can lead to unauthorized unlocks, false finality claims, arbitrary state anchoring, or any other privilege guarded only by successful MMR verification.

In the reviewed Hyperbridge call sites, this does **not** appear to be an immediately exploitable end-to-end issue. The reason is not that the library is safe, but that the current callers do not attach security-critical meaning to an empty verified leaf set.

In the message handlers, the leaf array is derived from the incoming batch length (`HandlerV1.sol#L156-L180`):

```
uint256 requestsLen = request.requests.length;
MerkleMountainRange.Leaf[] memory leaves = new
MerkleMountainRange.Leaf[](requestsLen);
...
bool valid = MerkleMountainRange.VerifyProof(root,
request.proof.multiproof, leaves, request.proof.leafCount);
...
for (uint256 i = 0; i < requestsLen; ++i) {
    host.dispatchIncoming(leaf.request, _msgSender());
}
```

If `requestsLen == 0`, the proof check may succeed trivially, but the dispatch loop also performs zero actions, so there is no immediate privilege gain from that empty success. Likewise, the BEEFY verification paths construct a non-empty leaf internally before calling the library ([BeeFYV1.sol#L225-L228](#)), so the empty-leaf forgery path is not reachable there.

The practical risk for Hyperbridge today is therefore best described as latent dependency risk: the bug is real, severe, and easy to exploit in the wrong integration shape, but the reviewed Hyperbridge flows do not currently expose that shape in a way that yields a concrete protocol break. The important caveat is that this safety margin is architectural, not cryptographic. Any future caller that treats `VerifyProof(..., [], ...) == true` as meaningful would become trivially vulnerable.

Mitigation

We recommend that the library should enforce full proof completeness and reject incomplete peak decomposition.

At minimum:

```
if (leafCount == 0) revert EmptyTree();
if (leaves.length == 0) revert EmptyLeaves();
...
if (subtreeLeaves.length == 0) {
    if (proofIter.data.length == proofIter.offset) revert
    ProofExhausted();
    _push(peakRoots, _next(proofIter));
}
...
if (proofIter.offset != proofIter.data.length) revert
UnconsumedProof();
if (peakRoots.offset != peakRoots.data.length) revert
UnconsumedPeaks();
```

If empty-leaf calls are intentionally supported for a specific non-inclusion API, they should not reuse the current inclusion verifier semantics. That behavior should be split into an explicit mode with clear invariants, rather than relying on partial peak reconstruction.

3.14 S1-16: Single-leaf shortcut returns leaf hash without proof

Attack scenario	For leafCount = 1, the verifier accepts leaves = [{0, root}] with arbitrary trailing proof data even though the only canonical proof is empty.
Classification	Proof canonicity / malformed proof acceptance
Component	EVM dependency / @polytope-labs/solidity-merkle-trees / MerkleMountainRange
Tracking	https://github.com/seunlanlege/hyperbridge/issues/16
Attack impact	Non-canonical proof accepted; no direct Hyperbridge authentication bypass identified
Severity	Low
Status	Mitigated

Background

In a one-leaf MMR, the only leaf is also the root. There are no sibling hashes to verify, so the only canonical proof shape is `proof.length == 0`.

`MerkleMountainRange.CalculateRoot` handles this degenerate case with a shortcut:

```
if (leafCount == 1 && leaves.length == 1 && leaves[0].index == 0)
{
    return leaves[0].hash;
}
```

Issue description

The shortcut returns `leaves[0].hash` without checking that `proof` is empty. As a result, these both verify:

```
VerifyProof(root, [], [{index: 0, hash: root}], 1) == true;
VerifyProof(root, [garbage, garbage], [{index: 0, hash: root}], 1)
== true;
```

The first case is correct. The second is non-canonical: the proof contains data that cannot be part of a valid one-leaf MMR proof.

This is not, by itself, an arbitrary leaf-authentication bypass. The attacker can only make verification pass when the supplied leaf hash equals the trusted root. In the reviewed Hyperbridge call sites, leaf hashes are derived from structured payloads before verification, such as `leaf.request.hash()`, `leaf.response.hash()`, or the encoded BEEFY MMR leaf. Exploiting this path against those callers would require a payload whose hash equals the trusted root.

Risk

The issue allows malformed, non-canonical proofs to be accepted, but no direct path was identified where this authenticates a forged Hyperbridge request, response, or BEEFY leaf. The main issue is latent dependency risk. A future caller that treats `leaf.hash` itself as authenticated payload data, or relies on `proof` bytes for replay protection, could make unsafe assumptions because arbitrary trailing proof data is accepted in this branch.

Mitigation

Require the proof to be empty in the shortcut, or remove the shortcut and add a general post-condition that all proof elements must be consumed before returning from `CalculateRoot`.

3.15 S0-10: Protocol fee rounds down to zero on small inputs

Attack scenario	A user splits an order into small legs to avoid protocol fees
Classification	Insecure Business Logic
Component	IntentGatewayV2 same-chain fills
Tracking	https://github.com/seunlanlege/hyperbridge/issues/10
Attack impact	Protocol revenue loss
Severity	Info
Status	Risk accepted

Background

`placeOrder` deducts a protocol fee from each input amount before recording escrow (`IntentGatewayV2.sol#L170-L182`):

```
uint256 originalAmount = order.inputs[i].amount;
if (originalAmount == 0) revert InvalidInput();
uint256 protocolFee = (originalAmount * protocolFeeBps) / 10_000;
uint256 reducedAmount = originalAmount - protocolFee;
```

Issue description

Integer division truncates toward zero. When `originalAmount * protocolFeeBps < 10_000`, `protocolFee` rounds to 0 and the user pays no protocol fee on that input leg. `originalAmount` is denominated in the token's smallest unit, so the human-readable threshold depends on token decimals.

- For `protocolFeeBps = 30` (a typical 0.3%), any leg with `originalAmount < 334` pays no fee.
- For `protocolFeeBps = 1` (0.01%), any leg with `originalAmount < 10_000` pays no fee.

A user can split a single intent into many small input legs to reduce their effective fee. For example, at 30 bps, 333 base-unit legs pay no protocol fee individually, while one larger leg would be charged normally.

Risk

The protocol loses fee revenue, not user funds. The economic damage is bounded by execution overhead since splitting into more legs eventually costs more in execution fees than the protocol fee saved.

The practical threshold is much higher for tokens with lower decimal precision because each zero-fee leg can represent more human-readable value; 0-, 1-, and 2-decimal tokens are the highest-risk shape, while 6-, 8-, and 18-decimal tokens are usually limited to dust-scale savings per leg.

At 30 bps, the largest zero-fee leg is 0.000333 USDC for a 6-decimal token or 3.33 tokens for a 2-decimal token.

Sophisticated solvers and high-frequency integrators have an incentive to optimize for this; retail users will not. Abuse would only be possible on chains with very cheap execution combined with the low-decimal tokens.

Mitigation

This can reasonably be accepted as a known rounding behavior if the gateway is intended for standard 18-decimal assets. In that case, document that protocol fees are rounded down per input leg and avoid supporting low-decimal high-value tokens.

If the protocol wants to eliminate zero-fee legs, use one explicit minimum-fee policy that rounds up and ensures every non-zero input pays at least one base unit when `protocolFeeBps > 0`. The tradeoff is that very small orders are overcharged relative to the configured percentage, so this should be an explicit product decision rather than a default security requirement.

4 Evolution suggestions

Security vulnerabilities in bridge infrastructure rarely occur in isolation. This engagement confirmed that the patterns underlying the original Hyperbridge exploit (silently ignored inputs, missing post-computation invariants, and divergence between reference and production implementations) recur across multiple components. Several of the new findings discovered during this audit share structural similarities with the original incident, despite affecting different contracts and different proof systems. This suggests that some root causes are systemic rather than localized: they reflect gaps in how verification code is developed, ported across languages, tested, and reviewed.

The mitigation suggestions in the previous section address each finding individually. The following recommendations target the shared development patterns (absent invariant documentation, inconsistent accounting models, and untested edge cases at proof-system boundaries) that allowed these defects to recur independently across both the verification and the settlement domains. They are grouped into technical measures, which target code and architecture, and procedural measures, which target the development and review lifecycle.

4.1 Technical recommendations

Adopt invariant-first development for all verification and escrow logic. The recurring pattern across this audit's findings is not insufficient testing, it is the absence of explicitly stated correctness invariants at the point where code is written. Each critical and high finding traces to a property that the Rust reference implementation enforces but that was never transcribed into the Solidity port: all proof elements must be consumed, leaf indices must be strictly increasing, escrow debits must match actual token receipts. This assessment recommends that Polytope establish a mandatory practice for all trust-boundary code (proof verifiers, escrow accounting, and authority-set validation), in which the required invariants are documented as part of the implementation specification, encoded as on-chain require assertions in the Solidity code, mirrored as regression properties in the test suite, and reviewed against the corresponding Rust reference before any PR is approved. Where a Solidity implementation lacks a Rust counterpart, the invariant documentation serves as the canonical reference. This practice does not replace external audits or differential fuzzing; it ensures that the most critical correctness properties are enforced at development time rather than discovered at audit time. The original Hyperbridge exploit and several findings in this report would not have survived a review process that required the developer to enumerate and assert the consumed-input invariant before merging.

Implement defense in depth for cross-chain operations. Cross-chain actions, including those originating from trusted sources such as governance, should be validated on the recipient gateway. The current design trusts that if the source chain says a message is legitimate, the destination accepts it without independent checks. Adding recipient-side validation (e.g., sanity bounds on withdrawal amounts, rate limiting on governance parameter changes, independent verification of state commitments) manages the risk of failure or compromise on the source infrastructure.

Maintain and audit dependencies, including unused components. Review and update all dependencies regularly, including components that are not actively used

in the current deployment. Stale or vulnerable dependencies that remain in the codebase can be reactivated by future changes or referenced by new code that places trust on them. At minimum, document any known untrusted or deprecated mechanisms so that future developers do not inadvertently rely on them.

4.2 Procedural recommendations

Test adversarial inputs, maintain regression tests, and cross-reference against reference implementations. The existing test suites primarily verify that well-formed inputs produce correct outputs. They do not systematically test how the code handles malformed, boundary, or adversarial inputs (e.g. empty arrays, duplicate entries, out-of-range values, oversized proofs, or missing fields). For every verification function, the test suite should include dedicated coverage for these cases. Each vulnerability discovered during this audit should additionally produce a regression test that fails on the vulnerable code and passes on the fix, ensuring the same class of bug cannot be reintroduced by future refactors. Beyond local testing, each Solidity implementation should be cross-referenced against its canonical reference: the Go Ethereum library for Patricia trie operations, the Polkadot Rust implementation for MMR logic, and the relevant protocol specification where one exists. Differential testing (and where feasible, differential fuzzing) between the Solidity code and the reference implementation is the most reliable way to surface divergences before they reach production.

Conduct regular security audits on logic changes before mainnet deployment. Any structural change to verification logic, escrow management, or consensus handling should undergo an independent security review before deployment to mainnet. This applies to refactors and feature additions equally: the risk profile of refactored code is comparable to new code.

Establish a bug bounty program with appropriate triaging. Implement a bug bounty program with clear scope, severity classification, and response commitments. If in-house triaging capacity is limited, use a third-party platform to manage submissions. Ensure that community-driven bug reports receive timely, structured assessment rather than ad-hoc review. The program should cover all deployed contracts in the verification and settlement stack.

Integrate static analysis into the development process. Continue using static analysis tools (Slither, etc.) as part of the CI pipeline, not just as periodic checks. Configure them to run on every pull request that touches verification or escrow logic. While static analysis would not have caught all findings in this audit, it catches common patterns like missing access control, unchecked return values, and reentrancy risks early.

Track known vulnerabilities and link fixes to issues. Maintain an internal vulnerability register that maps each known issue to its fix PR, its regression test, and its deployment status. This streamlines remediation tracking, future audit engagements, and bug bounty triage. When a new audit or bounty submission arrives, the register immediately shows whether the issue is known, fixed, or pending.

Improve off-chain monitoring for anomalous cross-chain activity. Consider enhancing off-chain monitoring to detect anomalous patterns in bridge operations:

unusual proof submission frequency, repeated authority indices in consensus proofs, large or rapid escrow releases, and state commitment values that deviate from expected ranges. Monitoring would not have prevented the original exploit, but it could have shortened the window between exploitation and detection.